

Grinder 设备端与安卓端

Python 保持路线与 C++ 热路径加速实施方案

核心结论

原方案具有较强参考价值，但不能原样执行。当前代码已完成设备端 32KB 接收、4KB 地图分片和部分有界队列；最高优先级应转为 Android 单写者、地图传输事务化、Super-LIO 健康门槛，以及安全与可观测性。C++ 只在基准确认热点后进入。

项目	内容
适用范围	Grinder20260901 / ROS1 Noetic 设备端；GrindingRobot Android APP
评审日期	2026-09-21
输出性质	代码现状核对 + 分阶段工程实施 + 测试与验收方案
推荐路线	先 Python/Kotlin/协议治理，再按火焰图结果选择性引入 C++
不在本次范围	不直接改动生产业务代码；不替换 Super-LIO、MST27 或完整调度器

内部工程评审稿 | 建议基线冻结后进入实施

文档导读

章节	回答的问题
1. 综合判断	原方案哪些正确、哪些已完成、哪些需要纠偏？
2. 当前代码事实	设备端和 Android 现在真正的瓶颈在哪里？
3. 目标架构	控制、事务与大数据如何隔离？
4. 路线一	不改 C++ 时具体改什么、先后顺序是什么？
5. 路线二	何时值得 C++ 化，边界和接口如何设计？
6. 实施计划	6 周如何落地、如何灰度和回滚？
7. 测试验收	功能、性能、故障、安全与稳定性如何证明？
8. 风险与交付物	上线门禁、责任边界和最终产物是什么？

阅读建议

管理评审重点看第 1、6、7 节；设备端开发重点看第 2、4、5 节；Android 开发重点看第 2.3、4.3 和 7.2 节。

1. 综合判断：有参考价值，但需按现状重排优先级

总体评价：约 70% 的方向成立，特别是“先测量、协议兼容优先、控制与大数据隔离、事务使用 operation_id、不要整体翻译 scheduler”这些原则。问题在于若干现状判断来自旧版本，且缺少 Android 发送并发、地图组包资源边界、安全门禁与定位质量门槛。

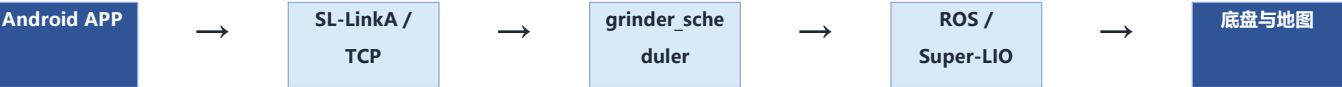
原方案主张	当前事实	判断	修订建议
设备 TCP 接收缓冲 4096B	设备端已使用 recv(32KB)	已完成/过时	不再作为设备端任务；Android 仍为 4096B，可调至 16–32KB
地图分片被压到 512B	设备端和 APP 当前均使用 4KB 上限	已完成/过时	先优化发送和组包，再 A/B 测试 16KB
批量发送地图	路径响应已有约 128KiB 合并写；地图仍逐片同步发送	部分成立	地图改为流式窗口 + 批量写，禁止一次构造所有分片
生产关闭逐帧详细 RX 日志	设备端已偏向头部日志；Android 发送成功仍构造全帧十六进制字符串	仍需完成	双端统一采样日志和统计日志
单一发送线程/队列	设备端有 send_lock；Android 普通发送、异步发送和心跳可并发 write+flush	高优先级	Android 先建单写者优先级 Channel
Livox 重复启动	当前 mapping launch 引入 Super-LIO，不是 Livox 驱动；base 脚本启动驱动	当前不成立	保留唯一所有权与节点冲突检查，避免回归
C++ 加速 SL 帧和 Protobuf	可能有效，但尚无火焰图证明；Python Protobuf/OpenCV 已有原生实现	有条件成立	达到热点阈值后再做，不先上 JNI/ 全量重写

决策

路线一必须先实施。路线二不是路线一失败后的补救，而是通过同一套基准证明某个 CPU 热点仍无法达标时，对该热点进行可回滚替换。

2. 当前代码事实与瓶颈

2.1 端到端链路



性能问题必须拆成五段：APP 排队与组包、TCP 传输、设备端解析与排队、ROS 业务执行、结果持久化/反馈。只看设备总 CPU 无法判断是否需要 C++。

2.2 设备端已具备的能力

- SL-LinkA 接收使用 32KB 缓冲，存在 send_lock、有界工作队列、控制响应并发限制与批量响应并发限制。
- 地图 MapRequest 分片默认及上限为 4096B，外层帧仍受 uint16 长度约束，4KB 处于安全区间。
- 路径批量响应已能把相邻完整协议帧合并为约 128KiB 的 TCP 写入，同时保持每帧可独立解析。
- Super-LIO 模式管理已有串行操作队列、启动服务/话题等待、残留建图节点 kill 兜底和地图文件基本校验。

2.3 仍然存在的主要问题

优先级	问题	代码证据/行为	主要影响
P0	Android 并发写同一 OutputStream	sendData、sendDataAsync、heartbeat 分别 launch 后 write+flush	帧交错、发送顺序不确定、频繁 flush、错误状态难追踪
P0	地图传输缺少事务标识和校验	MapChunk 只有 map_id/map_version/index/count，无 transfer_id、total_bytes、整图摘要	并发请求串图、旧分片污染、完整性不可证明
P0	生命周期和定位 READY 门槛不足	停止 kill 后不复核；initialpose 后任意 /lio/odom 即 ready	APP 误报成功、残留节点、低质量定位进入导航
P0	安全与鉴权基线不足	控制服务暴露、生产凭据/明文链路风险、底盘反馈闭环不足	未授权控制和故障状态下运动风险
P1	设备地图按请求重复全量处理	OccupancyGrid→NumPy→RGB→flip/resize→PNG；每次重做	大图首包延迟与 CPU 抖动
P1	设备地图先构造全部分片	outputs 列表持有全部序列化分片；随后逐片同步发送	内存峰值、接收线程占用、控制消息尾延迟
P1	Android 地图全内存重组	TreeMap 保存所有 ByteArray，完成后再写入 ByteArrayOutputStream 并复制	大地图内存峰值、OOM/GC 风险
P1	帧解析存在额外复制	Android CRC 重新拼接 header+payload，完成帧再 copy payload	高吞吐下对象分配和 GC

优先级	问题	代码证据/行为	主要影响
P1	测试覆盖缺口	缺 TCP 单写者、地图碎片/乱序/超时/恶意长度、断线重连压力测试	优化后容易出现隐性兼容回归

2.4 与通信性能同等重要的系统风险

- 三维建图内存并非主要由 Python 造成：Super-LIO 存在累计点云及关键帧全量复制路径，地图规模扩大时会直接推高 RSS。
- 定位链路还需四元数归一化、空目标点云防护、ICP/NDT fitness 与连续稳定帧门槛，否则“响应快”不等于“可以导航”。
- MST27 已有 C++ 热核；规划更需要超时、取消、内存预算和大图降采样，而不是重复翻译算法。
- 任何通信优化不得绕过急停、task_enable、速度清零、底盘反馈和控制仲裁。

3. 目标架构

3.1 三类业务、三套语义

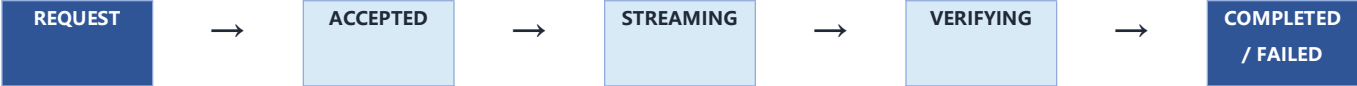
通道	典型消息	队列策略	确认语义	优先级
实时控制	急停、摇杆、速度、暂停	有界；速度 latest-wins；急停不可丢	接收/应用确认；超短超时	最高
业务事务	建图、保存、定位、删除、模式切换	可靠 FIFO；幂等键；禁止忙时静默丢弃	accepted→running→terminal	中
大数据	地图、轨迹、图片、历史记录	窗口化流式传输；可取消；限内存	分片进度+完整性校验	低

3.2 单连接发送模型



- Android 和设备端都只允许一个逻辑写者拥有 socket 输出流；其他协程/线程只提交不可变帧。
- 急停/控制可抢占尚未出队的大数据，但不能把一个已经开始写入的协议帧切开。
- 按字节数而不是仅按消息数限制队列，防止大地图分片耗尽内存。
- flush 由单写者按批次或延迟阈值执行；心跳也作为普通高优先级帧进入队列。

3.3 地图传输状态机



字段	用途	兼容策略
transfer_id	唯一标识一次下载，避免同 map_id 并发污	新增字段；旧端为空时退化为

字段	用途	兼容策略
	染	map_id+version
map_id + map_version	标识业务对象及不可变版本	保留现有字段
chunk_index / total_chunks	乱序重组与进度	保留并增加范围校验
total_bytes	提前设置资源上限，识别截断	新增可选字段
sha256 或 CRC32	整图完成后校验	优先 SHA-256；低端兼容 CRC32
encoding / geometry	PNG、栅格和坐标元数据	首片或 manifest 固定，后续不得变化
cancel / resume_token	取消和可选断点续传	先实现取消，断点续传后置

4. 路线一：Python、Kotlin 与协议优化（推荐先做）

4.1 基线与可观测性

1. 冻结测试场景：同一设备、同一雷达频率、同一地图文件、同一 APP 构建、同一日志级别。
2. 为每次请求生成 request_id/operation_id/transfer_id，记录 monotonic 时间戳。
3. 分别统计 APP 入队、APP 写完成、设备收帧、业务开始/结束、首片/末片发送、APP 校验/显示。
4. 每秒输出聚合指标：队列字节数、帧率、丢弃数、发送批次、p50/p95/p99、RSS、GC、CPU。
5. 采集设备 perf/py-spy、Android Perfetto/Studio Profiler 和网络 pcap，形成可复现基线报告。

4.2 设备端实施项

序号	改造项	实现要点	预期收益	风险
D1	地图响应接入 bulk TX	复用现有批量写；控制队列优先；队列按字节限额	减少 sendall/锁竞争，降低控制尾延迟	低
D2	分片生成器/窗口	不返回完整 outputs；按窗口生成、序列化、发送、释放	内存不随分片总数线性增长	中
D3	版本化地图缓存	key=map_id+version+encoding+尺寸/旋转参数；保存时生成；原子替换	消除重复 PNG 编码，缩短首包	中
D4	日志降频	生产默认只记录错误和每秒统计；禁止 payload JSON 化	降低 CPU 与磁盘抖动	低
D5	事务状态机	统一 operation_id 和终态；超时不得返回 success	消除 APP 假成功	中
D6	启动健康门槛	PID+ROS 节点+服务+话题频率+TF+定位质量	降低拉不起/关不掉和假 READY	中

4.3 Android 实施项

序号	改造项	实现要点	预期收益	风险
A1	单写者发送器	CoroutineScope+Channel；控制/事务/大数据优先级；所有 write/flush 唯一	先解决正确性，再降低系统调用	中

序号	改造项	实现要点	预期收益	风险
		入口		
A2	接收缓冲 16–32KB	保留任意分片/粘包兼容； 复用缓冲或 ByteBuffer	减少 read 与 copyOf 次数	低
A3	解析器少复制	增量 CRC；复用 payload buffer；限制最大帧/解析 预算	降低分配与 GC	中
A4	地图流式组包	按 transfer_id 写临时文 件；位图/BitSet 记录分 片；校验后原子发布	内存峰值稳定，支持大图	中
A5	资源与超时限制	total_bytes/chunks 上 限、空闲超时、重复片幂 等、并发传输数限制	防 OOM 与恶意/异常输入	低
A6	日志治理	禁止生产全帧 hex；采样+ 计数；敏感字段脱敏	明显减少 UI/IO 抖动	低

4.4 Super-LIO 生命周期与定位质量

- 停止：shutdown→等待→仅清理自有节点→再次确认；仍有残留则进入 TIMEOUT/ERROR 并返回残留节点清单。
- 启动：服务存在只是 STARTING；关键话题需满足频率与时间戳新鲜度，TF 需可变换且四元数有效。
- 定位：initialpose 必须与当前 map_id/version 关联；连续 N 帧配准质量、fitness、协方差和位姿跳变均达标后才 READY。
- 保存：PCD、PGM、YAML、元数据写入唯一临时目录，fsync 后原子切换；保存成功与定位启动成功分开反馈。
- 驱动所有权：保留 Livox 唯一启动入口；mapping launch 只拥有 Super-LIO/mapper/loop，启动前检查同名节点与端口。

5. 路线二：仅对高频热路径做 C++ 加速

5.1 启动条件

进入 C++ 路线的门槛

完成路线一后，在三轮可重复压测中，SL 帧扫描/CRC/组帧/Protobuf 编解码合计仍占设备单核 10%–15% 以上，或控制 p99/地图吞吐仍未达标且瓶颈明确位于 CPU，而不是网络、磁盘、PNG 编码或 ROS 业务，才进入 C++ 实现。

5.2 推荐边界

模块	建议	边界/接口	原因
SL 帧扫描、CRC、pack/unpack	优先候选	C++库/pybind11 扩展；输入 bytes，输出帧头+payload view	纯计算、高频、边界清晰
Protobuf 编解码	条件候选	只覆盖高频固定消息；协议生成物同源	Python Protobuf 本身可能已足够快
批量发送/地图分片	先 Python 实现	memoryview+生成器+sendall 批量窗口	主要收益来自模型改变，不是语言

模块	建议	边界/接口	原因
Android 解析器 JNI	暂不建议	仅在 Perfetto 证明解析持续热点后考虑	JNI 增加 ABI、崩溃和生命周期复杂度
scheduler_node.py	不整体重写	业务状态机保留 Python	规则复杂、变更频繁，翻译收益低
super_lio_mode_manager.py	不整体重写	ROS 编排保留 Python，补状态机和健康检查	瓶颈是语义和可靠性
MST27 / Super-LIO	继续现有 C++	补预算、健康门槛和内存上限	核心算法已经是 C++ 路径

5.3 C++ 组件工程要求

- 协议源唯一：只修改 sl_link.proto，再统一生成 Python、Android 与 C++ 代码；禁止手改生成文件。
- 兼容性：旧 Python codec 与新 C++ codec 进行字节级 golden corpus 对比，包含空载荷、最大载荷、未知字段和错误 CRC。
- 内存：固定上限、RAII、无未界定所有权；禁止把 payload 指针跨越输入缓冲生命周期。
- 故障隔离：扩展加载失败可通过 feature flag 回退 Python；C++ 异常不得越过 Python 边界。
- 发布：x86_64 与 aarch64 双平台 Release 构建，固定编译器/ABI，输出符号和版本信息。

6. 分阶段实施计划

阶段	时间	设备端	Android	出口条件
0 基线冻结	3-5 天	埋点、perf/py-spy、网络抓包	Perfetto、GC/内存、端到端时间戳	可重复基线误差≤10%
1 正确性优先	1 周	事务分类、地图走 bulk、停止复核	单写者、生产日志降频	10 万帧无交错；急停优先
2 地图传输	1-1.5 周	缓存、流式窗口、transfer manifest	流式落盘、校验、超时/取消	2/10/50MB 全通过故障注入
3 生命周期	1 周	operation 状态机、健康门槛、定位质量	分阶段状态 UI、重试/取消	无假成功、残留可诊断
4 性能优化	1 周	4KB/16KB A/B、复制优化	16/32KB 接收、解析少复制	目标 SLO 达标且 RSS 稳定
5 C++ 决策	2-3 天	审查火焰图与 CPU 占比	评估是否无需 JNI	满足门槛才立项
6 灰度上线	1 周	feature flag、10%→50%→100%	兼容旧协议与回滚	24h 稳定+现场回归

6.1 推荐任务拆分

6. 提交 1：只加指标和测试夹具，不改行为；形成基线报告。
7. 提交 2：Android 单写者和日志治理；保留旧实现 feature flag。
8. 提交 3：设备地图响应接入 bulk + 生成器窗口；协议不变。
9. 提交 4：协议新增 transfer_id/total_bytes/checksum/cancel；双端生成物同步。
10. 提交 5：Android 流式重组与设备版本化缓存。
11. 提交 6：Super-LIO operation 状态机、停止复核和定位质量门槛。
12. 提交 7：4KB/16KB A/B 与资源上限调优；决定是否需要 C++。

7. 测试与验收方案

7.1 测试分层

层级	设备端	Android	关键产物
单元	帧解析/CRC、队列策略、缓存 key、operation 状态机	parser、single-writer、chunk assembler、超时/上限	JUnit/Python 测试报告
协议契约	Python/C++生成帧	Kotlin 解析并反向生成	golden corpus + 字节级对比
集成	scheduler+模拟 socket+ROS stub	真机/模拟器连接设备模拟器	端到端时序和错误码
系统	RK3588+MID-360S+ROS 节点	目标安卓设备	现场地图/定位/导航记录
故障注入	节点挂死、磁盘满、服务超时、网络抖动	断网、切后台、低内存、重复请求	恢复与终态证明
稳定性	24h 建图/定位/地图传输循环	24h 前后台/重连/下载	RSS、线程、FD、队列趋势

7.2 必测用例矩阵

编号	场景	注入/规模	期望结果
T01	TCP 分片与粘包	每 1 字节、随机分片、100 帧一次 read	帧数/顺序/载荷完全一致
T02	坏帧恢复	错 CRC、错尾、随机噪声、截断	丢弃坏帧并在下一个 STX 恢复；无死循环/OOM
T03	并发发送	心跳+急停+状态请求+地图	线上字节流帧边界完整；急停不被大图阻塞
T04	地图乱序/重复/缺片	2、10、50MB；随机重排、重复、丢 1%	重复幂等；缺片超时；不发布半图
T05	并发地图请求	同 map 不同 version、同 version 两个 transfer	按 transfer_id 隔离，无串图
T06	地图完整性	篡改 1 字节、错误 total_bytes/hash	VERIFY_FAILED，临时文件清理
T07	连接中断	传输 25%/90%断网并重连	旧 transfer 终止；新请求可成功；无泄漏
T08	生命周期启动失败	缺服务、TF 无效、话题旧时间戳	终态 FAILED/TIMEOUT，不报告 READY
T09	生命周期停止失败	子节点忽略 shutdown	仅 kill 自有节点；复核失败则 ERROR 并列残留
T10	定位质量	空地图、低 fitness、协方差大、位姿跳变	拒绝 READY，APP 显示明确原因
T11	资源攻击/异常	伪造超大 total_chunks/total_bytes	立即拒绝，RSS 和磁盘受限
T12	24h 稳定性	周期保存/定位/下载/重连	RSS、FD、线程和队列无持续增长

7.3 建议验收指标（基线后确认）

指标	建议目标	测量口径
控制延迟	局域网并发传图时 p95 <100ms, p99	APP 提交到设备应用确认；另测底盘实际停机

指标	建议目标	测量口径
	<200ms；急停应用层 p99 <100ms	时间
地图吞吐	相对当前基线≥3×；稳定网络下有效吞吐 ≥5MB/s	不含首次地图生成；同时单列首包延迟
首包延迟	缓存命中 p95 <200ms；未命中单列 PNG 生成耗时	设备收到请求到首个 chunk 写出
正确性	10 万帧无交错、无错序；1000 次地图传输摘要 100%一致	双端记录 seq/transfer/hash
资源	单次传输额外 RSS <64MB 或<1.5×对象大小；队列有硬上限	RK3588 RSS + Android PSS/GC
生命周期	启动/停止/保存/定位无假成功；所有请求有唯一终态	故障注入 100 次
稳定性	24h 无崩溃；RSS、线程、FD 呈平台而非单调增长	每分钟采样并做趋势图

说明：以上是建议工程目标，不是当前实测结论。第一次基线完成后，应根据目标硬件、Wi- Fi 环境、地图尺寸和安全要求冻结正式 SLO。

7.4 性能测试组合

变量	取值
地图对象	2MB、10MB、50MB；PNG 高/低压缩率；栅格原始格式
chunk	4KB、16KB；32KB 仅探索，不直接生产默认
网络	无损；20/50/100ms RTT；1%丢包；5/20/100Mbps 限速；突发抖动
并发	仅地图；地图+状态；地图+手动控制；地图+急停；双地图请求
日志	生产级；诊断级（用于量化日志开销）
设备模式	建图、定位、导航；回环开/关需分别记录

8. 安全、兼容与回滚

- 控制面必须鉴权；生产凭据从仓库迁出到安全配置/密钥存储，外部链路采用 TLS 或受控网络隔离。
- 协议新增字段只做向后兼容扩展；旧端不认识字段时可忽略，新端检测旧能力后回退 4KB 和旧组包逻辑。
- 发送器、流式地图、C++ codec 均由独立 feature flag 控制，支持现场一键回退。
- 急停、速度清零、task_enable=false 和底盘反馈不受大数据限流影响；故障时默认安全。
- 灰度期间按设备维度保存版本、构建哈希、协议能力和关键指标；异常自动停止扩量。

9. 交付物与上线门禁

交付物	内容	完成定义
基线报告	CPU 火焰图、Perfetto、端到端延迟、吞吐、RSS/GC	场景可复现、原始数据可追溯
协议版本	proto、Python/Kotlin/C++生成物、能力协商、golden corpus	三端字节兼容测试通过

交付物	内容	完成定义
设备端改造	bulk 地图、流式窗口、缓存、operation 状态机、健康门槛	Python 单测+ROS 集成+RK3588 系统测试
Android 改造	单写者、流式组包、校验/取消/超时、日志治理	JUnit+instrumented+真机弱网测试
运维能力	指标、诊断开关、残留节点清单、版本/feature flag	现场可定位、可回滚
验收包	矩阵结果、24h 报告、缺陷清单、上线与回滚手册	P0 清零，P1 有明确豁免

上线门禁

未完成 Android 单写者、地图资源上限、生命周期无假成功、急停并发传图验证和 24 小时稳定性测试，不建议以“性能优化版”进入正式现场。

附录 A：关键代码位置与本次核对结论

仓库	文件	关注点
设备端	catkin_ws/src/grinder_scheduler/src/grinder_scheduler/sl_link_adapter.py	32KB 接收、队列、send_lock、bulk 发送；地图尚未接入 bulk
设备端	catkin_ws/src/grinder_scheduler/src/grinder_scheduler/scheduler_node.py	build_map_chunks: 4KB 上限、重复图像处理、完整 outputs 列表
设备端	catkin_ws/src/grinder_scheduler/src/grinder_scheduler/super_lio_mode_manager.py	串行生命周期、停止兜底、READY 门槛
设备端	third_party/sl_link/proto/sl_link.proto	MapChunk 缺 transfer_id/total_bytes/hash
设备端	catkin_ws/src/mapping/cloud_to_occupancy_grid/launch/mid360_mapping.launch	当前包含 Super-LIO/mapper/loop，不直接启动 Livox 驱动
Android	core/tcp/.../TcpService.kt	多线程直接写 OutputStream；4096B 接收缓冲
Android	core/tcp/.../TcpManager.kt	全帧 hex 日志；MapRequest 请求 4KB
Android	core/tcp/.../SlLinkManager.kt	MapChunk 按 map_id 全内存重组，无资源/超时/校验边界
Android	core/sllink/.../SlFrameParser.kt	逐字节解析正确但 CRC/载荷存在额外复制

附录 B：最终建议

建议批准路线一，暂缓路线二立项。先用 4-5 周把 Android 单写者、地图流式传输、协议事务字段、生命周期健康门槛和基准体系做好。这些改动同时提升速度、正确性和可维护性。完成后若设备端帧处理仍是明确热点，再用一个边界清晰、可回退的 C++ codec 扩展解决；不要先改写整个 scheduler，也不要先给 Android 引入 JNI。

— 文档结束 —